

ASCL HANDS-ON TRAINING · DAY 3 OF 4

ROS2

ROS 2 & LIMO Cobot Training

From Robot Concepts to Real-World Action

LAB	Automated Systems & Computing Lab (ASCL)
LAB LEADER	Prof. Ahmad Taher Azar

Dr Saim Ahmed
Trainer 1 · Day 1 & Day 3

Eng. Zeeshan Haider
Trainer 2 · Day 2 & Day 4



ASCL
Automated Systems & Computing Lab

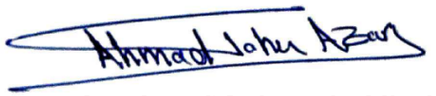
جامعة الامير سلطان
PRINCE SULTAN
UNIVERSITY



ACKNOWLEDGMENTS

This Training is supported by a research grant funded by the Research, Development, and Innovation Authority (RDIA), Kingdom of Saudi Arabia, with grant number 13382-psu-2023- PSNU-R-3-1-EI.

Prof. Ahmad Taher Azar



Full Professor, Leader of Automated Systems & Computing Lab (ASCL)

TRAINING AGENDA · 4 DAYS

Table of Contents

TRAINING 1 · DAYS 1–2 · FUNDAMENTALS

01 **Day 1 · What is ROS & ROS2?**

- Sense → Think → Act & the group-chat model
- Core concepts: nodes, topics, messages, services, actions
- turtlesim hands-on lab

02 **Day 2 · ROS2 Hands-On: CLI & First Nodes**

- The `ros2` CLI: node · topic · service · param
- Workspaces & packages
- First publisher & subscriber in Python

TRAINING 2 · DAYS 3–4 · ON THE ROBOT

03 **Day 3 · Meet the LIMO Cobot: Hardware & Bring-Up**

- Hardware, motion modes & sensors
- Connect & bring-up: NoMachine + chassis driver
- Drive & sense

04 **Day 4 · Applications: SLAM, Navigation & the Arm**

- SLAM mapping with Cartographer
- Navigation with Nav2
- myCobot 280 arm control

ROADMAP

Today's plan

01
LIMO Hardware
4 motion modes, sensors, the brain, safety rules. 15 min

02
Connect & Bring-Up
NoMachine → start `limo_base` → verify topics. 15 min

03
Drive & Sense
Publish `/cmd_vel`, drive it, see LiDAR in RViz2. 25 min

⚠️ Real robot today — start slow, keep feet clear, take turns.

BY THE END OF TODAY

You will be able to…

- Name the **4 motion modes** and read the robot's RGB light
- Identify each **sensor** and the topic it publishes
- Connect to the LIMO with **NoMachine** and start its chassis driver
- **Drive the robot** by publishing `Twist` to `/cmd_vel`
- See the LiDAR scan and odometry live in **RViz2**

01

Meet the LIMO Cobot

One chassis, four personalities, and a full sensor suite.

THREE THINGS IN ONE

What is the LIMO Cobot?

A desktop mobile robot by AgileX (partnered with Elephant Robotics), built **specifically for learning ROS**.

🚙 LIMO PRO base

4-wheel platform — drives, maps, navigates.

🦾 myCobot 280 M5 arm

6-joint arm — follow trajectory (Day 4).

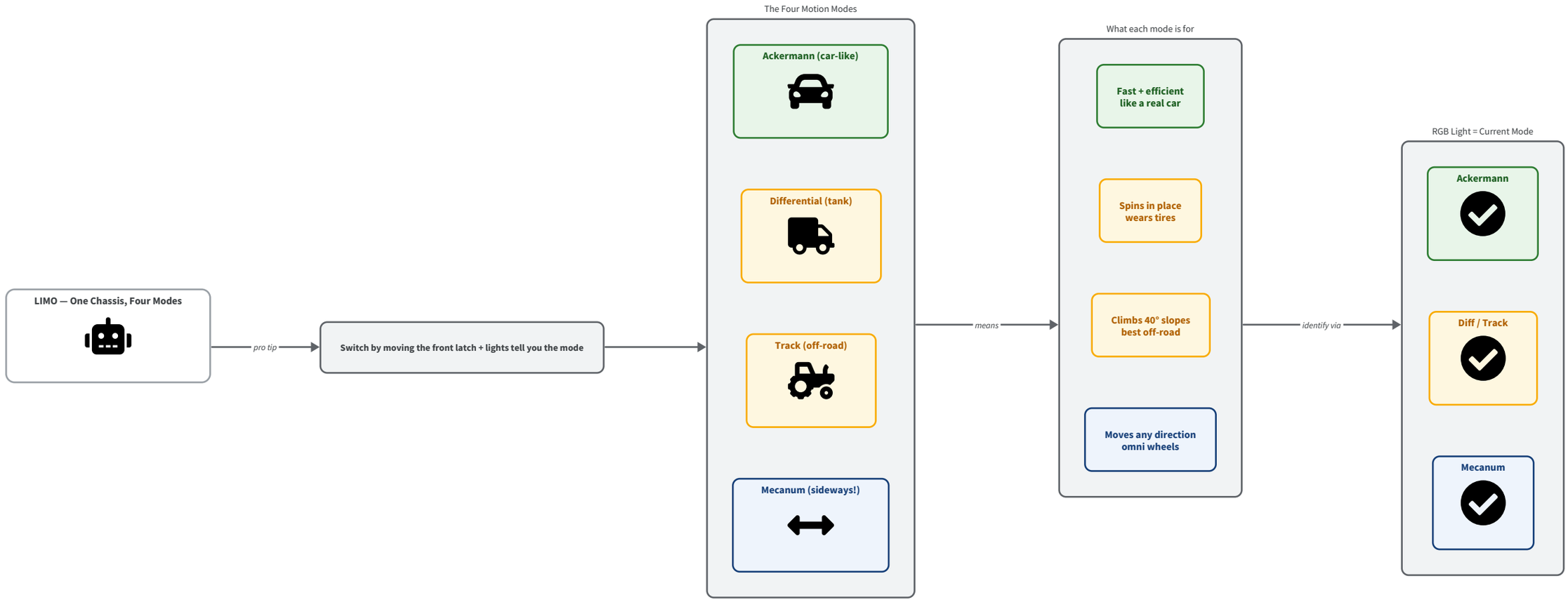
🧠 Jetson Orin Nano

NVIDIA computer running Ubuntu 20.04 + **ROS2 Foxy**.

Small, safe, open-source — everything transfers to any ROS2 robot in industry.

THE SUPERPOWER

Four motion modes, one chassis



THE FOUR MODES

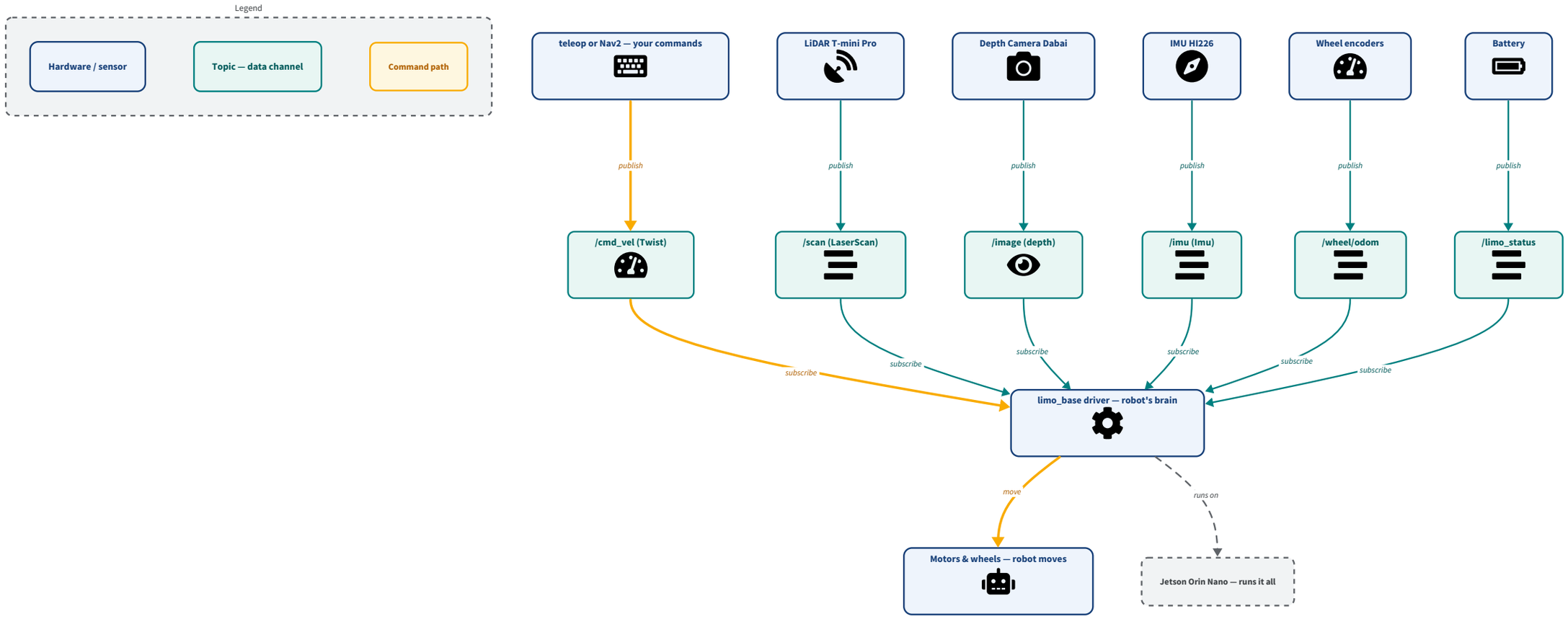
Which mode does what?

Mode	Light	Behaves like…	Best for
Ackermann 🚗	🟢 green	A real car (front wheels steer)	Fast, efficient driving
Differential 🚚	🟡 yellow	Wheels spin at different speeds	Spin in place, tight spaces
Track 🚜	🟡 yellow	A Tank (tracks on wheels)	Off-road, slopes (up to 40°)
Mecanum ↔️	🔵 blue	Omni wheels with rollers	Moving sideways & diagonally

⚠️ Never switch modes while moving; don't spin differential in place for long — it wears the tires.

THE EYES & EARS

The sensors — and their topics



SENSOR TO TOPIC

Each sensor is a node publishing a topic

Sensor	Senses	Topic	Analogy
📡 LiDAR (EAI T-mini Pro)	360° distances, up to 12 m	`/scan`	Eyes for seeing walls
📷 Depth camera (ORBBEC)	3D depth + color	`/image`	3D vision
🧭 IMU (HI226)	Acceleration + orientation	`/imu`	Inner-ear balance
⚙️ Wheel encoders	Wheel rotation	`/odom`	"How far have I traveled?"
🔋 Battery	Voltage level	`/limo_status`	Fuel gauge

The LIMO is just a bigger, realer turtle — same nodes, same topics.

THE BRAIN

Jetson Orin Nano + ROS2 Foxy

A credit-card-sized NVIDIA computer with a GPU, running **Ubuntu 20.04 with ROS2 Foxy** — the exact distro from Days 1–2.

Why it transfers

- Your Day-2 talker/listener runs unchanged
- `ros2 node list` shows real nodes: `limo_base`, `lidar`, `camera`...

What it is

- The onboard computer
- Reachable over Wi-Fi via NoMachine
- The robot's "brain" you'll remote into

READ THESE

Safety rules

Rule	Why
🔋 Charge the battery (12.6 V, ~2.5 h)	Robot stops at low voltage
⚠️ Don't charge while driving	Safety + battery life
🛑 Know where the power switch is	You can kill power instantly
🚧 Keep people & feet clear of the path	LiDAR can't see everything
🌡️ -10°C to 40°C, not waterproof	Keep it indoors and dry
🔒 Don't modify hardware alone	Warranty + safety

02

Connect & Bring-Up

Remote into the robot's brain and start it.

HOW YOU REACH THE ROBOT

Two ways in

Method	What it is	When to use
🖥️ **NoMachine** remote desktop	See & click the robot's screen from your laptop	**Today** — you can see everything
💻 SSH (text only)	Command-line access	Later, for scripting

Instructor confirms: robot on same Wi-Fi, powered on, in **differential mode** (yellow light).

CONNECT

NoMachine, step by step

1. Install **NoMachine** (free) from nomachine.com/download
2. Open it — it scans the network and finds the LIMO
3. Double-click the LIMO (or add a connection by IP)
4. Login when prompted:

Login

`Username: agilex`

`Password: agx`

You now see the robot's Ubuntu desktop — **you're inside the robot's brain.** 🧠

> Can't find the robot? Both must be on the same Wi-Fi — or connect by IP from the instructor.

START THE BRAIN

Launch the chassis driver

Everything the robot does starts with the **chassis driver** — a node that talks to motors and sensors over serial. It's called `limo_base`.

LIMO desktop terminal — leave running

```
ros2 launch limo_base limo_base.launch.py
```

A **launch file** (Day 2!) starts several nodes at once — the drivers publishing odometry, IMU, and status.

VERIFY

Is the brain alive?

Second terminal — detective skills

```
ros2 topic list
```

`/cmd_vel`
speed commands (you'll send these)

`/imu` · **`/odom`**
orientation · distance traveled

`/limo_status`
battery & errors

Check one is publishing

```
ros2 topic echo /limo_status  # battery voltage 🎉
```

03

Drive & Sense

Take the wheel — every command makes a real robot move.

BEFORE YOU DRIVE

Setup & LiDAR note

- Terminal 1 is still running `limo_base`.
- Robot in **differential mode**, area clear .

> **LiDAR note:** `/scan` appears only when the LiDAR driver runs — `limo_base` alone does **not** start it. If `ros2 topic list` shows no `/scan`, start it in its own terminal:

Foxy — base + LiDAR together

```
ros2 launch limo_bringup limo_start.launch.py
```

STEP 1

Command it with one message

The "hello world" of driving — send ONE speed command:

Terminal 2

```
ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.1}, angular: {z: 0.0}}"
```

The LIMO **moves forward a tiny bit** and stops. You just drove a robot with a message — same mechanism as your Day-2 talker, but the subscriber is real motors.

STEP 2

Drive smoothly with the keyboard

Terminal 2

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

The keys

key map

```
   u    i    o
   j    k    l
   m    ,    .
```

Controls

- **i** forward · **,** backward
- **j** left · **l** right · **k** stop
- **Shift** = faster · **q/z** = adjust speed

> Everyone gets a turn (30 s): slow rectangle, then a figure-eight. Gentle — its job today is teaching, not racing.

STEP 3

See what the robot sees — RViz2

Terminal 3

```
rviz2
```

If empty, click **Add** → **By topic**, then add:

LaserScan

from `/scan` — room walls as a ring of points

Odometry

from `/odom` — arrow of robot position

TF

the robot's coordinate frames

Drive slowly while watching — the wall points move with the robot. The LiDAR sees 360°.

STEP 4

Read the LiDAR's raw data

Terminal 3

```
ros2 topic echo /scan
```

A `LaserScan` message: an array of `ranges` (distances) from `angle_min` to `angle_max` — one distance per angle step around the robot.

> Walk to the side of the robot and echo again — the distances on that side shrink. **You're reading the robot's eyes.**

STEP 5 · BONUS

Watch the odometry

While someone drives slowly

```
ros2 topic echo /odom
```

`pose.pose.position.x` grows as the robot moves forward — its own estimate of "how far have I traveled."

This is **crucial on Day 4**, when the robot must navigate a map.

THE CLASSIC PROBLEM

"I published /cmd_vel but it didn't move"

#	Check	Fix
1	Is `limo_base` still running?	Restart if it crashed
2	Right motion mode?	**Differential** (yellow light), not Ackermann
3	Speed too small?	`linear.x: 0.1` is gentle — try `0.3`
4	Battery?	`ros2 topic echo /limo_status` — low voltage blocks motion
5	Two publishers fighting?	`Ctrl+C` the teleop node

RECAP

What you did → what it proves

What you did	ROS2 concept it proves
`ros2 topic pub /cmd_vel …`	Publishing a **Twist** — the robot moved
`teleop_twist_keyboard`	A **node** translating keys into `/cmd_vel`
RViz2 LaserScan	Visualizing the `/scan` topic — 360° view
`ros2 topic echo /scan` / `/odom`	Reading raw messages — eyes & odometer

THE BIG TAKEAWAY

Nothing here was new

Topics, messages, nodes, echo — all from Days 1–2. The only difference: **the subscriber is a real robot.**

You already know how to drive a robot — you just didn't know it.



Quick Check

Five minutes. Pass ≥ 70% — answers revealed on each slide.

Q1 · Which motion mode behaves like a real car?

A) Mecanum

B) Track

C) Ackermann

D) Differential

▸ Show answer

Q2 · Which RGB color means Mecanum mode?

A) Green

B) Yellow

C) Red flashing

D) Blue

▸ **Show answer**

Q3 · Which sensor publishes `/scan`?

A) Depth camera

B) LiDAR

C) IMU

D) Wheel encoders

▸ **Show answer**

Q4 · To move forward at 0.1 m/s, you publish...

A) A `LaserScan` on `/scan`

B) A `Pose` on `/turtle1/pose`

C) A `Twist` on `/cmd_vel` with `linear.x = 0.1`

D) A `String` on `/chatter`

▸ **Show answer**

Q5 · What does `ros2 launch limo_base limo_base.launch.py` do?

A) Opens RViz2

B) Starts the chassis driver nodes (odometry, IMU, status)

C) Starts the LiDAR only

D) Shuts down the robot

▸ **Show answer**

Q6 · `ros2 topic pub --once /cmd_vel …` will...

A) Move forward continuously forever

B) Move forward a short distance, then stop

C) Turn left

D) Do nothing

▸ **Show answer**

TROUBLESHOOTING SCENARIO

Robot won't move — what do you check?

`limo_base` is running. You publish a `Twist` — nothing. Name the 4 checks, in order.

The order that works

1. Motion mode (differential, yellow)
2. Speed too small → try 0.3
3. Low battery → `/limo_status`
4. Two publishers → kill teleop

Fixes

- Switch the latch
- Raise `linear.x`
- Charge / swap battery
- `Ctrl+C` the leftover teleop

DAY 3 COMPLETE ✅

Tomorrow: the robot thinks.

Day 4 is the grand finale — the LIMO **maps a room it's never seen** (SLAM), **drives itself to a goal** (Nav2), and **picks up an object** with its arm.